

MEDIATEK

MT8676 Hypervisor System User Manual

Version: 1.1
Release date: 2024-11-19

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-11-04	Kai Peng	• Official release
1.1	2024-11-19	Kai Peng	• Added the descriptions of full virtualization L+A+L. • Added Section 1.2.3 Thyp sdk Prebuilt File • Added Section 0 • Add Shared Folders • Added Section 1.5.1 Switch USB

Table of Contents

Version History	2
Table of Contents	3
List of Figures	3
List of Tables	3
1 Hypervisor System	5
1.1 Abbreviations	5
1.2 System Architecture	5
1.2.1 Project	6
1.2.2 Code Structure	7
1.2.3 Thyp sdk Prebuilt File	7
1.3 Compilation and Flashing	9
1.3.1 Build Server	9
1.3.2 Compile Instructions	10
1.3.3 Software Composition	12
1.3.4 Flashing Software	13
1.4 Customization	16
1.4.1 Adjust Virtual Machine Memory	16
1.4.2 Adjust Virtual Machine CPU	16
1.4.3 Add Shared Folders	17
1.5 Debugging Techniques	18
1.5.1 Switch USB	18
1.5.2 ADB	18
1.5.3 Query Virtual Machine Status	19
Exhibit 1 Terms and Conditions	20

List of Figures

Figure 1-1. Full virtualization L+A+L architecture	6
Figure 1-2. Partial virtualization L+A+L architecture	6
Figure 1-3. Thyp SDK prebuilt files directory structure	8
Figure 1-4. Download SP_Flash_Tool	14
Figure 1-5. Load flash.xml	15
Figure 1-6. Flashing software	15
Figure 1-7. L+A+L memory layout	16

List of Tables

Table 1-1. Abbreviations	5
Table 1-2. Yocto Hypervisor project	7
Table 1-3. Hypervisor code	7
Table 1-4. Thyp SDK prebuilt files explanation	8
Table 1-5. Yocto build server requirements	9
Table 1-6. Android build server requirements	10

Table 1-7. SOS image file12

Table 1-8. UOS Android image file13

Table 1-9. UOS T-Box image file.....13

Table 1-10. adb shell instruction18

1 Hypervisor System

The MT8676 Hypervisor SDK consists of three main components: Yocto, Android, and Thyp SDK. This chapter primarily introduces the system content related to Yocto and Android with the Hypervisor. For an introduction to the Yocto single system, please refer to the document *MT8676_Yocto_System_User_Manual*, and for the Android single system, please refer to the document *MT8676_Android_System_User_Manual*. This chapter includes content on system architecture, compilation and flashing, customization, debugging tools and techniques.

1.1 Abbreviations

Table 1-1. Abbreviations

Abbreviation	Explanation
SOS	Service OS
UOS	User OS
nebula	The GoldenRiverTek hypervisor micro kernel

1.2 System Architecture

The MT8676 Hypervisor supports two architectures. One is the full virtualization L+A+L architecture, as shown in [Figure 1-1](#). The other is the partial virtualization L+A+L architecture, as shown in [Figure 1-2](#). Both architectures consist of four main components. The first is the GoldenRiverTek Hypervisor and the Nebula microkernel. The second one is SOS, which acts as the host and primarily runs the hypervisor and backend virtualization drivers. The third one is UOS Android, which acts as the guest and primarily runs the frontend virtualization drivers and some passthrough hardware drivers. The fourth one is UOS T-Box, which acts as the guest and primarily runs the T-Box and GPS drivers. Both SOS and UOS T-Box are Linux systems, compiled through the Yocto build framework. UOS Android is an Android system.

The main difference between the full virtualization architecture and the partial virtualization architecture is that modules such as APU, GPU, Camera, and Codec are changed from being virtualized to being directly passed through to UOS Android.

MT8676 has only one USB hardware, so it can only be passed through to one operating system at a time. For the full virtualization architecture, it is passed through to the SOS by default. For the partial virtualization architecture, it is passed through to UOS Android by default. For the method to switch the USB, please refer to [Section 1.5.1](#).

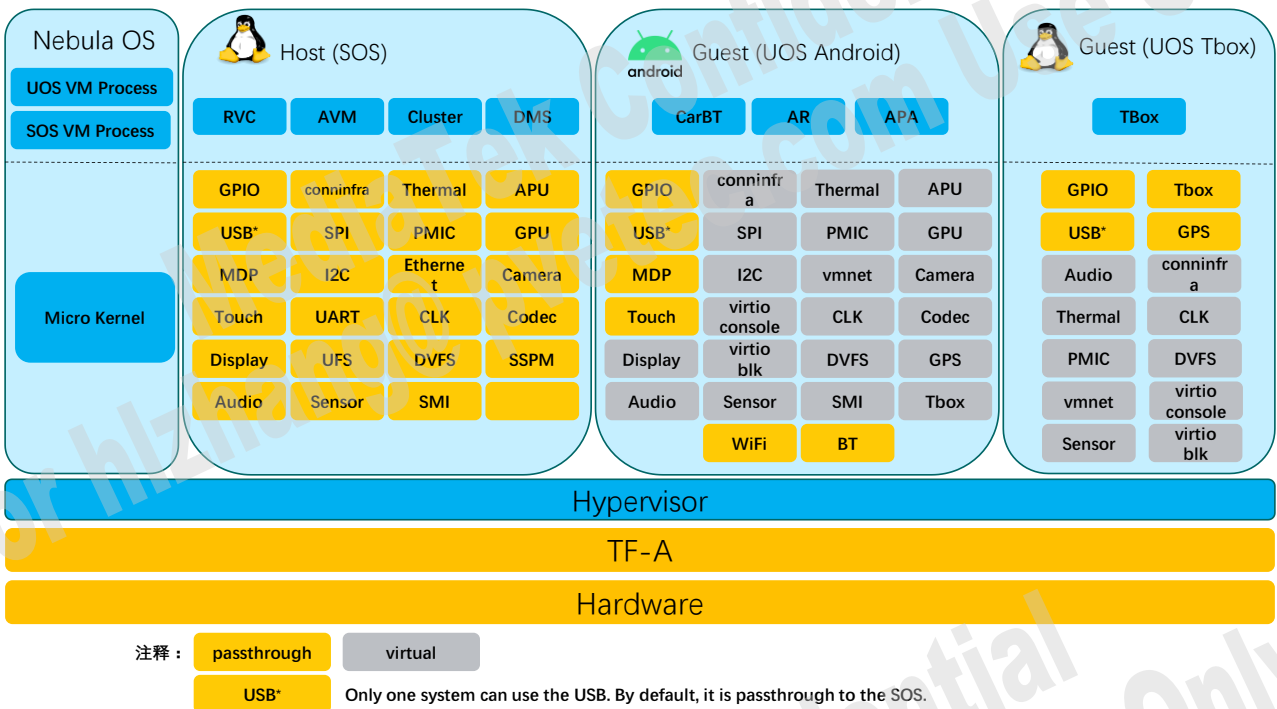


Figure 1-1. Full virtualization L+A+L architecture

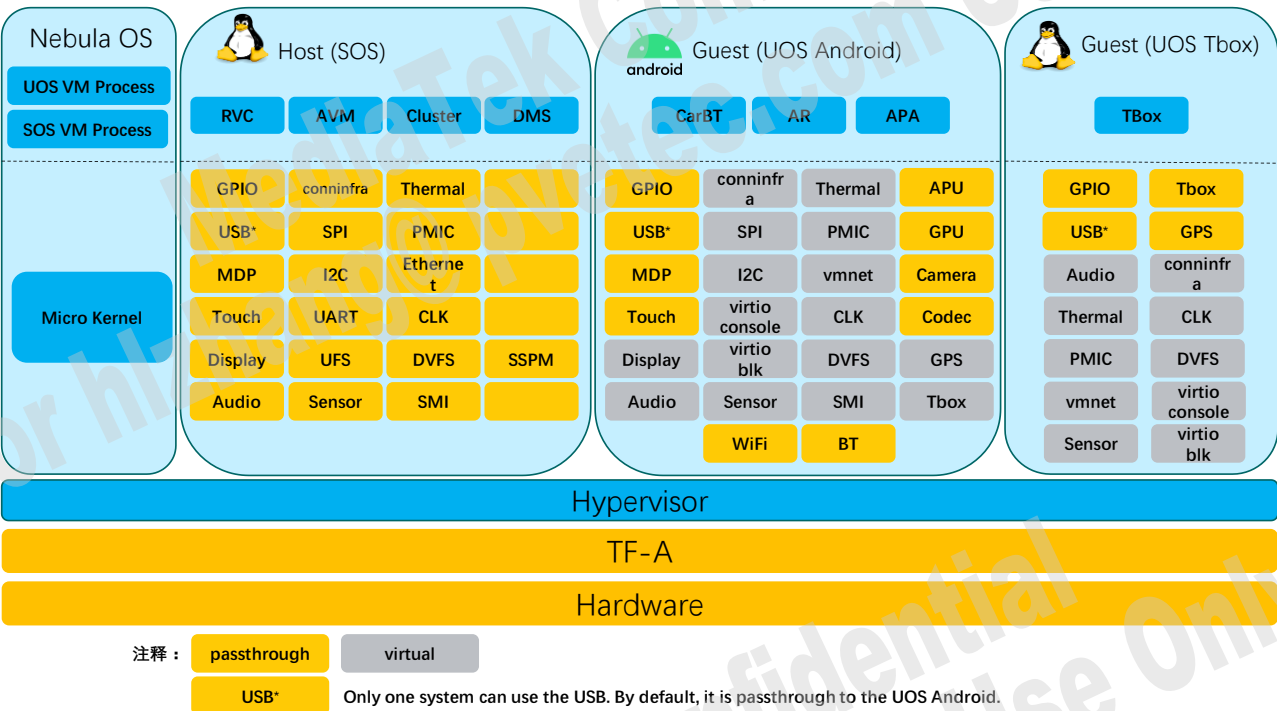


Figure 1-2. Partial virtualization L+A+L architecture

1.2.1 Project

Table 1-2 lists the Yocto project and Android project provided by the MT8676 Hypervisor SDK, with a total of two sets. It is important to note that the Yocto project and Android project correspond one-to-one and cannot be mixed. Yocto requires

two projects. In the following text, SOS refers to auto8676p1_64_hyp or auto8676p1_64_hyp_sos. UOS Android refers to auto8676p1_64_bsp_vm or auto8676p1_64_bsp_vm_tbox. UOS T-Box refers to auto8676p1_64_uos_tbox.

Table 1-2. Yocto Hypervisor project

Yocto (SOS)	Yocto (UOS T-Box)	Android (UOS Android)	Explanation
auto8676p1_64_hyp	auto8676p1_64_uos_tbox	auto8676p1_64_bsp_vm	Full Virtualization.
auto8676p1_64_hyp_sos	auto8676p1_64_uos_tbox	auto8676p1_64_bsp_vm_tbox	Partial Virtualization.

1.2.2 Code Structure

The MT8676 Hypervisor is an extension based on the single Yocto system and single Android system, with some Hypervisor-related code added on top of the single system. Therefore, the code structure is exactly the same. For the code structure of the single system, please refer to the documents *MT8676_Yocto_System_User_Manual* and *MT8676_Android_System_User_Manual*. The full virtualization architecture and the partial virtualization architecture share the same codebase.

Table 1-3. Hypervisor code

System	Path	Explanation
Yocto	meta/meta-mediatek-mt8676-hyp	The MT8676 Yocto virtualization layer includes some configuration files and bb files related to virtualization.
Yocto	src/kernel/modules/mt8676/virt/grt	Virtualization drivers on the Yocto side
Yocto	prebuilt/hypervisor/grt_mt8676	The GoldenRiverTek thyp sdk prebuilt file
Android	vendor/mediatek/kernel_modules/virt	Virtualization drivers on the Android side
Android	device/mediateksample/auto8676p1_64_bsp_vm	Configuration files for the Android auto8676p1_64_bsp_vm project
Android	device/mediateksample/auto8676p1_64_bsp_vm_tbox	Configuration files for the Android auto8676p1_64_bsp_vm_tbox project

1.2.3 Thyp sdk Prebuilt File

The prebuilt files for the Thyp SDK are stored in the `prebuilt/hypervisor/grt_mt8676` directory of the Yocto codebase. These mainly include the Nebula virtual machine image, the virtual machine manager program, and virtualization configuration files. The directory structure is shown in [Figure 1-3](#). The `auto8676p1_64_hyp` directory contains files related to the full virtualization architecture, the `auto8676p1_64_hyp_sos` directory contains files related to the partially virtualization architecture, and the `auto8676p1_64_uos_tbox` directory contains configuration files for UOS T-Box. For the function and description of each file, please refer to [Table 1-4](#). Except for `gz.img`, all other files are installed in the root file system of SOS.

```

├── auto8676p1_64_hyp
│   ├── gz.img
│   ├── nbl_vmm
│   ├── uos_alps_bootloader_lk2.pb.txt
│   ├── uos_alps_pv8676.lua
│   └── vm_srv_cfg.pb.txt
├── auto8676p1_64_hyp_sos
│   └── ...
├── auto8676p1_64_uos_tbox
│   ├── uos_tbox_bootloader_lk2.pb.txt
│   └── uos_tbox_pv8676.lua
├── check_gl.sh
├── gpu_server
├── libluajit.so
├── libvmm.so
├── nblruncmd
├── nbl_vm_ctl
├── nbl_vm_post_android.sh
├── nbl_vm_post_tbox.sh
├── nbl_vm_pre.sh
├── nbl_vm_srv
├── nbl_vm_srv_post.service
├── nbl_vm_srv.service
├── nc-vsock
├── NebulaProprietary
├── query_input_event.sh
├── symbols
│   └── ...
├── video_server
├── virtiofsd
├── vm_audio_cfg.pb.txt
└── vm_audio_shared_irq.pb.txt

```

Figure 1-3. Thyp SDK prebuilt files directory structure

Table 1-4. Thyp SDK prebuilt files explanation

File or Directory	Explanation	Path
gz.img	The firmware for the GoldenRiverTek Nebula virtual machine. The configuration file for SOS is packaged in gz.img.	gz_a partition.
nbl_vmm	Virtual Machine Manager. For SOS, a UOS is a nbl_vmm process.	/usr/bin
vm_srv_cfg.pb.txt	Virtual machine service configuration file.	/vendor/etc/hyper_android
nbl_vm_srv	Virtual machine service program. It reads the configuration from the vm_srv_cfg_8676.pb.txt file and starts the nbl_vmm process based on the configuration.	/usr/bin
nbl_vm_srv.service	A systemd service, used to start the nbl_vm_srv process.	/usr/lib/systemd/system
nbl_vm_pre.sh	Used to execute some tasks before the virtual machine starts.	/vendor/etc/hyper_android

File or Directory	Explanation	Path
nbl_vm_srv_post.service	A systemd service, used to execute some tasks after the virtual machine has started.	/usr/lib/systemd/system
nbl_vm_post_android.sh	Used to execute some tasks after UOS Android has started.	/vendor/etc/hyper_android
nbl_vm_post_tbox.sh	Used to execute some tasks after UOS T-Box has started.	/vendor/etc/hyper_tbox
nbl_vm_ctl	Virtual machine control program. It can start and stop virtual machines, as well as query some basic information about the virtual machines.	/usr/bin
uos_alps_bootloader_lk2.pb.txt	Bootloader file for UOS Android. It is used to configure bootloader-related information.	/vendor/etc/hyper_android
uos_alps_pv8676.lua	Configuration file for UOS Android virtual machine. It is used to configure the virtual machine's CPU, memory, interrupts, etc.	/vendor/etc/hyper_android
uos_tbox_bootloader_lk2.pb.txt	Bootloader file for UOS T-Box.	/vendor/etc/hyper_tbox
uos_tbox_pv8676.lua	Configuration file for UOS T-Box virtual machine.	/vendor/etc/hyper_tbox
gpu_server	GPU virtualization host-side handler.	/usr/bin
video_server	vcdec virtualization host-side handler.	/usr/bin
virtiofsd	virtio filesystem backend program.	/usr/bin
symbols	Symbol files for storing nbl_vmm, nbl_vm_srv, nbl_vm_ctl, and other files, used for debugging.	N/A

1.3 Compilation and Flashing

1.3.1 Build Server

Table 1-5 lists the requirements for the build host to compile MT8676 Yocto 5.0. Table 1-6 lists the requirements for the build host to compile MT8676 Android.

Table 1-5. Yocto build server requirements

Item	Requirement
Disk Space	Not less than 300GB
Memory	Not less than 32GB
Distribution Version	Ubuntu 20.04, 22.04, etc., for specific versions refer to Supported Linux Distributions ¹ .

¹ <https://docs.yoctoproject.org/ref-manual/system-requirements.html#supported-linux-distributions>

Item	Requirement
Build host tools	Refer to Required Packages for the Build Host ² .
Git	Version 1.8.3.1 or higher
tar	Version 1.28 or higher
Python	Version 3.8.0 or higher
GNU Make	Version 4.0 or higher

Table 1-6. Android build server requirements

Item	Requirement
Disk Space	At least 250GB
PC Memory	At least 24GB
Version	Ubuntu 18.04 (Recommended)
Tools which should be installed for compilation	Refer to http://source.android.com/source/initializing.html
Git	>1.9.1
Shell	>4.4.1 (Ubuntu18.04 default built-in version) (Recommended)
Perl	>5.26.1 (Ubuntu18.04 default built-in version) (Recommended)
Python	Python 2.7.17&Python 3.6.9 (Ubuntu18.04 default built-in version) (Recommended)
GNU Make	>4.1 (Ubuntu18.04 default built-in version) (Recommended)

1.3.2 Compile Instructions

For the L+A+L architecture, you need to compile Yocto auto8676p1_64_hyp_sos, Yocto auto8676p1_64_hyp_uos_tbox, and Android auto8676p1_64_bsp_vm. The compilation instructions are as follows. Since some environment variables will be set during the compilation process, it is recommended to use three different terminals to compile the two Yocto projects and Android separately to avoid interference. To compile two different Yocto projects from the same codebase, a parameter is passed to the oe-init-build-env script to specify the name of the build directory. Additionally, the build target for auto8676p1_64_hyp_uos_tbox is mtk-core-image-auto8676-uos.

The Yocto auto8676p1_64_hyp_sos build artifacts are located in the build-sos/tmp/deploy/auto8676p1_64_hyp_sos directory. The Yocto auto8676p1_64_hyp_uos_tbox build artifacts are located in the build-uos/tmp/deploy/auto8676p1_64_uos_tbox directory. The Android build artifacts are located in the out/target/product/auto8676p1_64_bsp_vm_tbox/merged directory.

After the compilation is complete, please copy the image files listed in [Table 1-8](#) and [Table 1-9](#) to the SOS build directory.

```
# terminal 1, Yocto (SOS) auto8676p1_64_hyp
cd path/to/yocto-codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-hyp/conf/templates/auto8676p1_64_hyp
```

² <https://docs.yoctoproject.org/ref-manual/system-requirements.html#required-packages-for-the-build-host>

```
source meta/poky/oe-init-build-env build-sos
bitbake mtk-core-image-auto8676
```

```
# terminal 2, Yocto (UOS Tbox) auto8676p1_64_uos_tbox
cd path/to/yocto-codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-
hyp/conf/templates/auto8676p1_64_uos_tbox
source meta/poky/oe-init-build-env build-tbox
bitbake mtk-core-image-auto8676-uos
```

```
# terminal 3, Android auto8676p1_64_bsp_vm
cd path/to/android-codebase
python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py --run
full_auto8676p1_64_bsp_vm-userdebug
```

For the partial virtualization architecture, you need to compile Yocto (SOS) auto8676p1_64_hyp_sos, Yocto (UOS T-Box) auto8676p1_64_uos_tbox, and Android (UOS Android) auto8676p1_64_bsp_vm_tbox. The compilation instructions are as follows.

The compilation outputs for SOS are located in the build-sos/tmp/deploy/images/auto8676p1_64_hyp_sos directory. The compilation outputs for UOS T-Box are located in the build-uos/tmp/deploy/images/auto8676p1_64_uos_tbox directory. The compilation outputs for Android are located in the out/target/product/auto8676p1_64_bsp_vm_tbox/merged directory.

After the compilation is completed, please copy the image files listed in [Table 1-8](#) and [Table 1-9](#) to the SOS build directory.

```
// terminal 1, Yocto (SOS) auto8676p1_64_hyp_sos
cd path/to/yocto-codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-
hyp/conf/templates/auto8676p1_64_hyp_sos
source meta/poky/oe-init-build-env build-sos
bitbake mtk-core-image-auto8676
```

```
// terminal 2, Yocto auto8676p1_64_uos_tbox
cd path/to/yocto-codebase
export TEMPLATECONF=${PWD}/meta/meta-mediatek-mt8676-
hyp/conf/templates/auto8676p1_64_uos_tbox
source meta/poky/oe-init-build-env build-tbox
bitbake mtk-core-image-auto8676-uos
```

```
// terminal 3, Android auto8676p1_64_bsp_vm_tbox
cd path/to/android-codebase
python vendor/mediatek/proprietary/scripts/releasetools/split_build_helper.py --run
full_auto8676p1_64_bsp_vm_tbox-userdebug
```

1.3.3 Software Composition

Table 1-7 lists the image files for SOS, Table 1-8 lists the image files for UOS Android, and Table 1-9 lists the image files for UOS T-Box.

Table 1-7. SOS image file

File Name	Partition	Explanation
MT6897_Android_scatter.xml	N/A	Partition table, compiled by Yocto ptgen-v2.bb.
download_agent/DA_BR.bin	N/A	Download agent, used for flashing. Compiled by collect.bb.
bl2.img	preloader_a preloader_b	Yocto Bootloader, compiled by Yocto lk2.bb.
modem.img	modem_a	Modem firmware, compiled by Yocto modem.bb.
spmfw.img	spmfw_a	System power management firmware, compiled by collect.bb.
mcf_ota.img	mcf_ota_a	Compiled by Yocto collect.bb.
pi_img.img	pi_img_a	Compiled by Yocto collect.bb.
dpm.img	dpm_a	Compiled by Yocto collect.bb.
scp.img	scp_a	System Companion Processor firmware, compiled by Yocto tinysys-scp.bb
ccu.img	ccu_a	Camera Control Unit firmware, compiled by collect.bb
vcp.img	vcp_a	VCP firmware, compiled by tinysys-vcp.bb.
sspm.img	sspm_a	Secure System Power Manager firmware, compiled by collect.bb.
mcupm.img	mcupm_a	Power/performance manager firmware, compiled by collect.bb.
gpueb.img	gpueb_a	GPU firmware, compiled by tinysys-gpueb.bb.
apusys.img	apusys_a	APU firmware, compiled by tinysys-apusys.bb.
gz.img	gz_a	GoldenRiverTek nebula hypervisor firmware, compiled by nbl-vmm-hyp.bb.
connsys_bt.img	connsys_bt_a	BT firmware, compiled by collect.bb.
connsys_wifi.img	connsys_wifi_a	WIFI firmware, compiled by collect.bb.
connsys_gnss.img	connsys_gnss_a	GNSS firmware, compiled by gps-fw_1.0.0.bb.
logo.img	logo_a	Logo, compiled by makelogo.bb.
audio_dsp.img	audio_dsp_a	Audio DSP firmware, compiled by collect.bb.
system.ext4	system	Yocto root filesystem, compiled by mtk-core-image-auto8676.bb
userdata.ext4	yocto-userdata	Yocto /data partition, compiled by mkusrdata.bbclass.
bl2-an.img	bl2-an	Android bootloader, compiled by bl2-an.bb.
yocto-boot.img	yocto-boot_a	Linux kernel in FIT format, including Kernel img, dtb, and initramfs. Compiled by linux-mtk-extension_6.1.bb. The initramfs is compiled by core-image-minimal-initramfs.bb.

Table 1-8. UOS Android image file

File Name	Partition	Explanation
vbmeta.img	vbmeta_a	The meta data for the system and vendor layers, compiled by Android.
vbmeta_system.img	vbmeta_system_a	The meta data for the system layer, compiled by Android.
vbmeta_vendor.img	vbmeta_vendor_a	The meta data for the vendor system layer, compiled by Android.
boot.img	boot_a	The Android kernel image, compiled by Android.
vendor_boot.img	vendor_boot_a	The Android ramdisk ko image, compiled by Android.
init_boot.img	init_boot_a	The Android init image, compiled by Android.
dtbo.img	dtbo_a	The Android dtb overlay image, compiled by Android.
tee.img	tee_a	The ATF, compiled by Android.
super.img	super	The Android super image, compiled by Android.
userdata.img	userdata	The Android userdata partition, compiled by Android.

Table 1-9. UOS T-Box image file

File Name	Partition	Explanation
bl2-tbox.img	bl2-tbox_a	UOS T-Box bootloader, compiled by Yocto lk2.bb.
boot_uos_tbox.img	boot_uos_tbox_a	The UOS T-Box Linux kernel in fit format, including the kernel image and dtb, compiled by linux-mtk-extension-uos_6.1.bb.
system_uos_tbox.ext4	system_uos_tbox	The UOS T-Box root filesystem, compiled by Yocto mtk-core-image-auto8676-uos.bb.
userdata_uos_tbox.ext4	userdata_uos_tbox	The UOS T-Box /data partition, compiled by Yocto mkusrdata.bbclass.

1.3.4 Flashing Software

The flashing software requires a Type-C cable and the SP_Flash_Tool tool. If you do not have the SP_Flash_Tool tool, please visit the [Online](https://online.mediatek.com/apps/tool/)³ website to download it. After opening the webpage, search for SP_Flash_Tool and download the latest version, as shown in [Figure 1-4](#). After the download is complete, extract the downloaded archive.

³ <https://online.mediatek.com/apps/tool/>

Home

Filter

Tool Name : SP_Flash_Tool

Product Line :

4N Release :

Query

Download

Download Cart

Total 2 items < 1 > 20 / page

☐	Name	Version	Release Date	Product Line	Download
☐	SP_Flash_Tool	v2404.00.000	2024-06-07	ALPS	   
☐	SP_Flash_Tool	v2216.00.000	2024-06-07	ALPS	   

Figure 1-4. Download SP_Flash_Tool

First, connect the platform to the computer using a Type-C cable. Then double-click on SP_Flash_Tool_V6/SPFlashToolV6.exe to open the burning tool. For the first step, select the Download-XML file, click the choose button in the top right corner, and select the download_agent/flash.xml file under the software package. For the second step, select Format All + Download from the dropdown menu below the Download button, as shown in Figure 1-5. For the third step, ensure that the platform is powered off. For the fourth step, click the Download button. For the fifth step, hold down the download key (KPCOLO, SW907) without releasing to power on the platform, and the burning will automatically start, as shown in Figure 1-6. At this point, you can release the download key.

If you encounter an error popup, first power off the platform and click the stop button to exit the download mode. Make sure the platform is completely powered off before attempting to burn again.

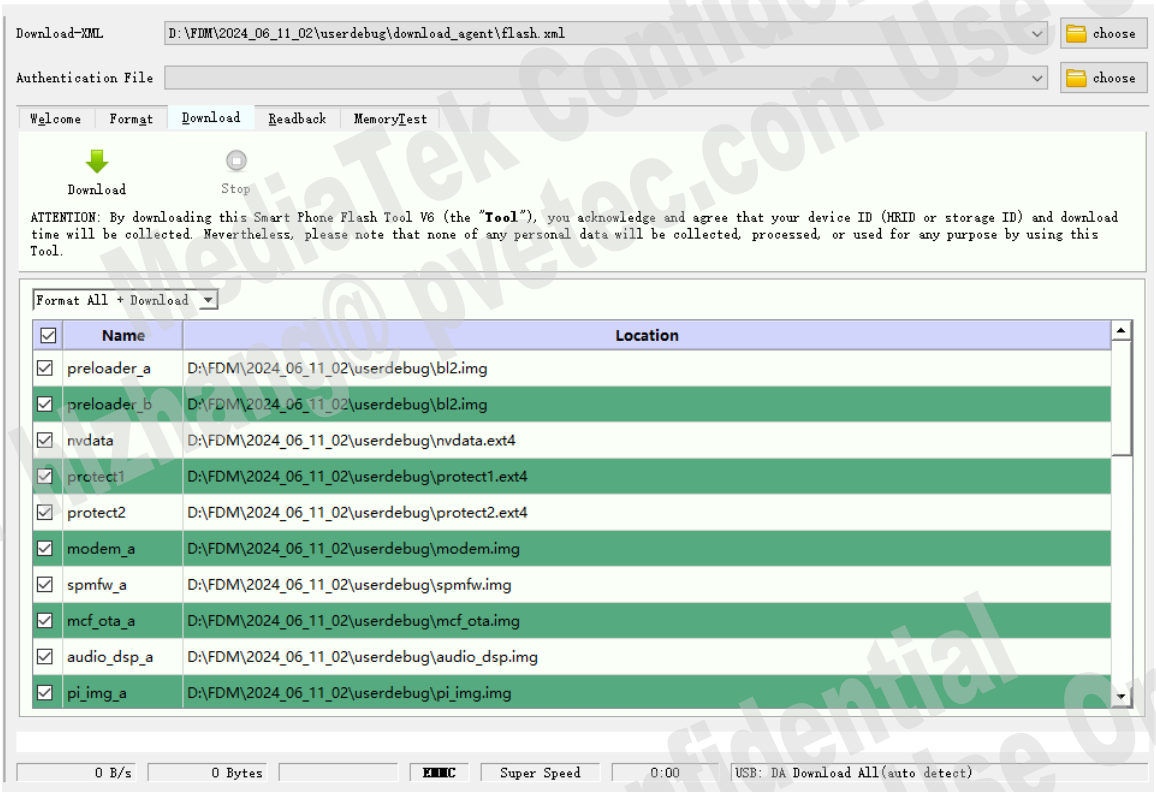


Figure 1-5. Load flash.xml

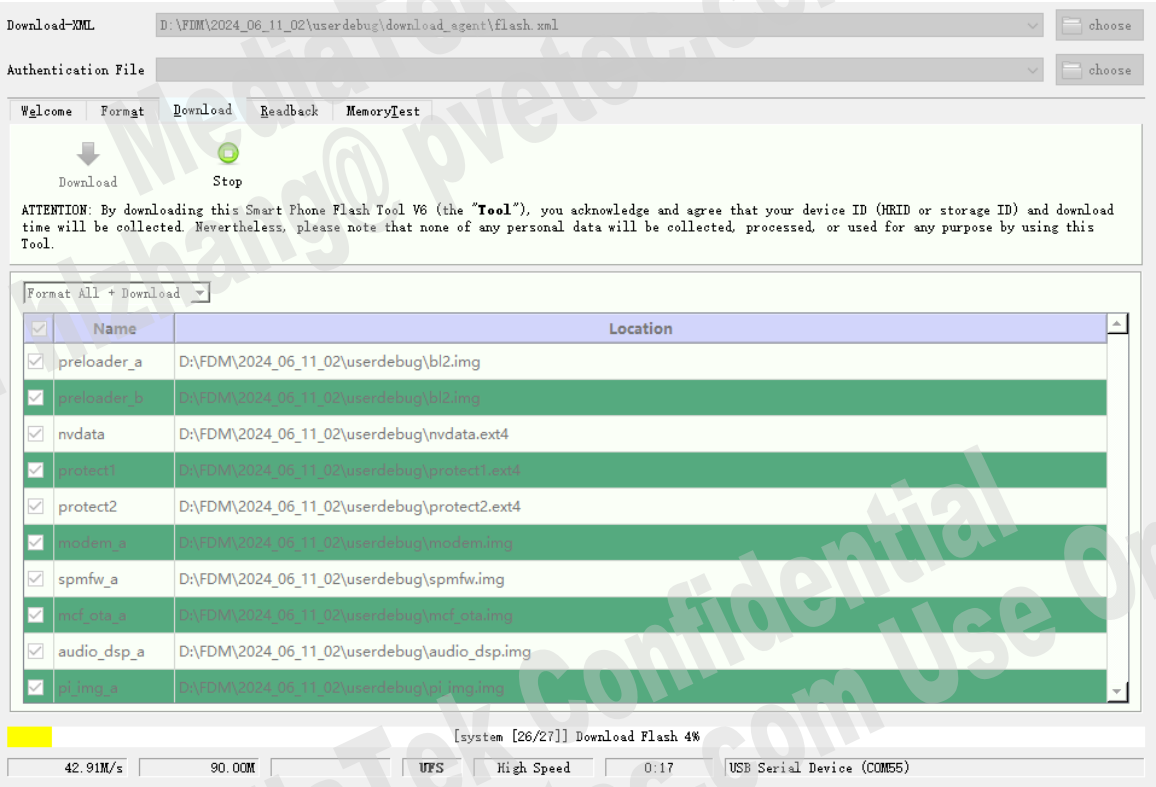


Figure 1-6. Flashing software

1.4 Customization

1.4.1 Adjust Virtual Machine Memory

Figure 1-7 shows the memory layout of the L+A+L architecture. The configuration file meta/meta-mediatek-mt8676-hyp/conf/common/memory-size.inc in the Yocto codebase determines the memory size for each operating system. The memory size for SOS is determined by the variable SOS_DRAM_SIZE, with a default value of 6 GB. The memory size for UOS T-Box is determined by the variable UOS_TBOX_DRAM_SIZE, with a default value of 1 GB. UOS Android uses all the remaining memory and is not controlled by any specific variable. On the public platform, the total physical memory is 16GB, so the memory for UOS Android is 9GB. It is important to note that some modules will request reserved memory in the bootloader, and this memory is allocated from SOS_DRAM_SIZE. To ensure the normal operation of SOS, it is recommended that SOS_DRAM_SIZE not be less than 2 GB.

```
# SOS Yocto memory size
SOS_DRAM_SIZE = "0x180000000"

# for 3os
UOS_TBOX_DRAM_SIZE = "0x40000000"
```

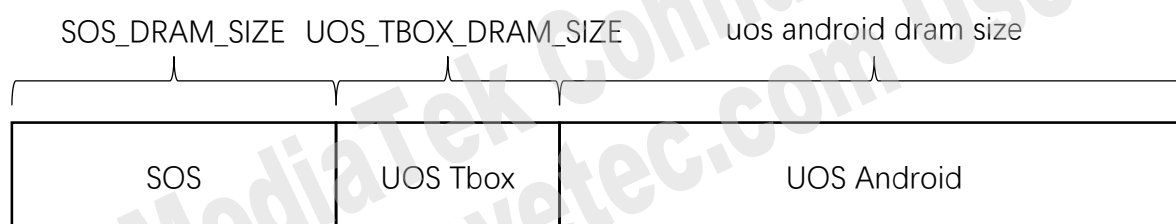


Figure 1-7. L+A+L memory layout

1.4.2 Adjust Virtual Machine CPU

The number of CPUs for the virtual machine and their binding relationship with the physical CPUs are determined by the virtual machine's configuration file. Please refer to Table 1-4 for the paths of the virtual machine configuration files. In the following example configuration, a total of 8 vCPUs are allocated to the virtual machine. The array pCPUs determines the binding relationship between physical CPUs and virtual CPUs. The index represents the vCPU, and the value represents the pCPU. For instance, vCPU0 is bound to pCPU3, vCPU1 is bound to pCPU1, vCPU2 is bound to pCPU2, vCPU3 is bound to pCPU0, and so on. Please ensure that the number of CPUs equals the length of the pCPUs array; otherwise, the virtual machine will fail to start. On the MT8676 platform, there are three CPU clusters in total: pCPU0~pCPU3 belong to the same cluster, pCPU4~pCPU6 belong to another cluster, and pCPU7 belongs to the third cluster. Do not bind CPUs across different clusters, as this will prevent the CPUs from performing optimally.

```
-- cpus
uos_config:setCpus(8)
--*****

-- physical-cpus
```

```
Pcpus = {3, 1, 2, 0, 4, 5, 6, 7}
for i = 1, #Pcpus do
    uos_config:setBincpus(Pcpus[i])
end
--*****
```

1.4.3 Add Shared Folders

If you want to share a folder from SOS to UOS, you can achieve this through the virtio filesystem. The following are the implementation steps. For the configuration file paths mentioned below, please refer to [Table 1-4](#).

Step 1: Modify `vm_srv_cfg.pb.txt` to add the virtiofsd backend program for the target UOS. For example, the following sample code adds a virtiofsd backend program to UOS T-Box. The `shared-dir` parameter is used to specify the shared folder, and the `socket-path` and `tag` parameters will be used later.

```
vmrecords {
  name: "tbox"
  .....

  backends {
    exec: "virtiofsd"
    args: "--socket-path=/tmp/virtiofs_tbox_socket0"
    args: "--shared-dir=/mnt/vendor/nvdata"
    args: "--tag=tbox_nvdata"
    args: "--sandbox=none"
    args: "--modcaps=+sys_admin"
  }
}
```

Step 2: Modify the UOS configuration file and add the following code. The first parameter is the tag, and the second parameter is the socket path. Please refer to the values from Step 1 for these two parameters. Here, take UOS T-Box as an example, and you need to modify the `uos_tbox_pv8676.lua` file.

```
uos_config:setFs("tbox_nvdata", "/tmp/virtiofs_tbox_socket0")
```

Step 3: Power off and restart the entire system. After UOS has fully started, enter the UOS shell. Execute the following command to test the mount. The third parameter `tbox_nvdata` is the tag parameter set in Step 1. If the mount is successful, it indicates that the folder sharing is successful. If it fails, go back and check Step 1 and Step 2.

```
mkdir -p /tmp/test_mount
mount -t virtiofs tbox_nvdata /tmp/test_mount
```

Step 4: Set up auto-mount on startup. For UOS Android, modify the `device/mediateksample/<project>/init.project.rc` file. For UOS T-Box, modify the `meta/meta-mediatek-mt8676-hyp/recipes-core/base-files/auto8676p1_64_uos_tbox/fstab` file.

1.5 Debugging Techniques

For debugging techniques related to Yocto, please refer to the document *MT8676_Yocto_System_User_Manual*. For debugging techniques related to Android, please refer to the document *MT8676_Android_System_User_Manual*. This section will only introduce debugging techniques related to the Hypervisor.

1.5.1 Switch USB

The MT8676 has only one USB hardware, so only one operating system can use the USB at a time. For a fully virtualized architecture, the USB is passed through to the SOS by default. For a partially virtualized architecture, the USB is passed through to the UOS Android by default. You can execute one of the following commands to switch the USB to the specified system. The commands can be executed on any system. If executing them on UOS Android, please switch to the root user. After executing the command, you need to power off and restart the system for the settings to take effect.

```
# switch to SOS
sysenv_test write USB_SWITCH Y
# switch to UOS Android
sysenv_test write USB_SWITCH A
# switch to UOS Tbox
sysenv_test write USB_SWITCH T
```

1.5.2 ADB

The operating system with the USB uses the physical adb. The other two systems use network ADB. [Table 1-10](#) lists the commands to enter the ADB shell for each system. For example, in a fully virtualized architecture, the USB is assigned to the SOS by default, so you need to use the command in the first row.

Table 1-10. adb shell instruction

USB	SOS	UOS Android	UOS T-Box
SOS	adb -d shell	adb -d forward tcp:7666 tcp:6666 adb connect 127.0.0.1:7666 adb -s 127.0.0.1:7666 shell	adb -d forward tcp:7667 tcp:6667 adb connect 127.0.0.1:7667 adb -s 127.0.0.1:7667 shell
UOS Android	adb -d forward tcp:7665 tcp:6665 adb connect 127.0.0.1:7665 adb -s 127.0.0.1:7665 shell	adb -d shell	adb -d forward tcp:7667 tcp:6667 adb connect 127.0.0.1:7667 adb -s 127.0.0.1:7667 shell
UOS T-Box	adb -d forward tcp:7665 tcp:6665 adb connect 127.0.0.1:7665 adb -s 127.0.0.1:7665 shell	adb -d forward tcp:7666 tcp:6666 adb connect 127.0.0.1:7666 adb -s 127.0.0.1:7666 shell	adb -d shell

1.5.3 Query Virtual Machine Status

On the SOS, execute the command `cat /sys/guest_os/android/pm_state` to query the status of UOS Android.

Execute the command `cat /sys/guest_os/tbox/pm_state` to query the status of UOS T-Box.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.